A Configurable Dynamic-Fee Hook Framework for Uniswap v4 Liquidity Pools

***Abstract*—Uniswap v4 makes swap fees programmable through hooks, but implementing and comparing alternative fee policies still requires compatible Solidity contracts, permission-aware deployment, pool initialization, market-data inputs, and repeatable analysis. This paper presents a configurable development and evaluation framework that integrates these tasks. Its browser interface allows developers to select a supported dynamic-fee policy, configure its parameters, inspect the generated Solidity source, and export a self-contained Foundry project. The framework deploys hook-enabled pools on a local Forge network and evaluates them using Binance-derived price and volume profiles, Chainlink-compatible mock feeds, and simulated retail and arbitrage flow. We compare five dynamic policies with four static-fee configurations in 29,160 experimental cells spanning three token pairs, 72 one-day windows per pair, three gas-price scenarios, and five retail-demand levels. Dynamic-policy outcomes are paired with a predeclared 30-basis-point static baseline under identical market conditions and summarized using bootstrap confidence intervals. No dynamic policy provides an unconditional advantage over the baseline. Conditional improvements occur in high-volatility, low-gas settings, with the best-performing policy depending on the retail-to-arbitrage flow composition; higher gas prices remove these gains. The framework therefore provides reproducible infrastructure for constructing, deploying, and comparing Uniswap v4 dynamic-fee hooks, while the evaluation shows that their economic benefits are conditional rather than universal.**

***Index Terms*—automated market makers, decentralized exchanges, dynamic fees, Uniswap v4, smart-contract hooks, liquidity provision**

I. Introduction

Decentralized finance (DeFi) provides exchange, lending, derivatives, and asset-management services through publicly executable smart contracts [1]. Within this ecosystem, decentralized exchanges (DEXs) allow users to trade without a custodial order-matching intermediary. Many DEXs rely on automated market makers (AMMs), which execute swaps against liquidity reserves according to a deterministic function [2], [3]. Uniswap popularized the constant-product AMM and later introduced concentrated liquidity, allowing liquidity providers (LPs) to allocate capital within selected price ranges [4], [5]. These mechanisms enable permissionless market creation, but they also expose LPs to inventory changes and adverse-selection risk.

An AMM pool price changes when swaps modify the pool state. When an external market price moves before the pool price, arbitrageurs can trade against the stale pool until the remaining discrepancy no longer covers fees and transaction costs. This process changes the composition and marked value of the LP position. Impermanent loss compares the LP position with simply holding the deposited assets, whereas loss-versus-rebalancing measures the cost of passive rebalancing at stale prices relative to a continuously rebalanced benchmark [6]. Concentrated liquidity adds another trade-off: a narrow range can earn a larger share of fees while active, but stops earning fees when the market leaves the range and may require repositioning [7].

Swap fees compensate LPs for providing liquidity and bearing these risks [8]. A static fee must, however, serve different market conditions. A high fee can discourage ordinary trading during calm periods, while a low fee may provide insufficient compensation during volatile or arbitrage-intensive periods. Dynamic-fee mechanisms instead adjust the fee according to information such as recent price movement, volatility, trade direction, or trade size. In this paper, *dynamic* refers only to the swap fee. The hook does not directly set the asset price: the pool price remains determined by liquidity, the AMM state, and executed swaps.

Prior studies analyze this fee-setting problem from several perspectives. Fritsch [9] examines optimal fees for constant-function market makers. Lebedeva *et al.* evaluate block-adaptive, deal-adaptive, and oracle-informed asymmetric policies [10]; Baggiani *et al.* derive dynamic fees through stochastic control [11]; and Campbell *et al.* study fee selection when an AMM competes with a centralized exchange [12]. Agent-based research further considers heterogeneous traders, latency, arbitrage, and maximal-extractable-value searchers [13]. These studies motivate adaptive fees and provide candidate policies, but an economic rule alone does not provide a deployable Uniswap implementation or a common environment in which several policies can be compared consistently.

Uniswap v4 provides the protocol mechanism needed for such implementations. A hook is an external smart contract attached to a pool when the pool is initialized and invoked at specified points of its lifecycle. For a dynamic-fee pool, the hook can update the stored LP fee or return a fee override for an individual swap [14], [15]. Implementing a usable hook nevertheless requires more than translating a fee equation into Solidity. Developers must encode callback permissions, obtain a permission-compatible hook address, deploy Uniswap core and periphery contracts, initialize a pool, provide liquidity and price inputs, execute swaps, and collect comparable outputs. Reimplementing this surrounding infrastructure for every policy reduces reproducibility and can confound differences in fee logic with differences in deployment or simulation code.

This paper addresses that engineering gap with a configurable development and evaluation framework for Uniswap v4

dynamic-fee hooks. Through a browser interface, a developer selects a supported hook template, configures its parameters, and inspects the resulting Solidity source. The framework can export a self-contained Foundry project, deploy the selected hook and a new hook-enabled pool on a local Forge network, provide Chainlink-compatible mock price feeds, execute historical-data-driven scenarios, and present the resulting metrics and figures. The generated source remains visible and independently executable; the configurator supports development and experimentation rather than replacing compilation, testing, or security review.

The framework is evaluated through a controlled comparison of five dynamic policies and four static-fee configurations. The experiment uses 2024 Binance-derived price and volume profiles for ETH/SHIB, ETH/USDC, and USDC/USDT. For each pair, 72 one-day windows represent low-, medium-, and high-volatility conditions. Every policy is evaluated under three gas-price scenarios and five retail-demand levels using a simulated market containing ordinary users and a gas-aware arbitrageur. This produces 29,160 experimental cells. Dynamic-policy outcomes are paired with a predeclared 30-basis-point static baseline under the same pair, historical window, gas price, demand level, and random inputs.

The comparison finds no dynamic policy that outperforms the static baseline unconditionally. Conditional advantages nevertheless appear during high-volatility periods when gas prices are low. Which policy benefits depends on the balance between ordinary-user and arbitrage volume, while the additional gas consumed by dynamic hooks removes these gains as gas prices rise. The results therefore do not support universal superiority of dynamic fees; instead, they demonstrate that their effectiveness depends on market conditions, order-flow composition, and implementation cost.

The contributions of this work are:

- A configurable Uniswap v4 development framework that combines dynamic-fee hook templates, browser-based parameter selection, generated-source inspection, and self-contained Foundry project export.
- A reusable deployment and execution path that initializes hook-enabled pools on Forge, integrates Chainlink-compatible mock feeds, replays historical market conditions, and measures contract-level gas and pool outcomes.
- A reproducible factorial evaluation comprising 29,160 experimental cells across five dynamic policies, four static fees, three token pairs, three volatility regimes, three gas-price scenarios, and five retail-demand levels.
- Empirical evidence that dynamic-fee hooks provide no unconditional advantage over a well-chosen static fee, but can improve LP outcomes under specific high-volatility and low-gas conditions.

The evaluation is conducted on a local EVM using historical market observations and modeled trader behavior. It does not reproduce live oracle operation, mempool competition, strategic routing, or public-chain liquidity dynamics. Its purpose is to demonstrate a reproducible method for constructing and comparing deployable fee policies and to identify the conditions under which their benefits appear or disappear.

II. Background and Related Work

A. AMM Fees and LP Exposure

For reserves x and y, a constant-product AMM maintains

$$xy = k, \tag{1}$$

apart from fees, rounding, and protocol accounting. If a trader supplies Δx and the proportional fee is f, the simplified output is

$$\Delta y = y - \frac{xy}{x + (1-f)\Delta x}. \tag{2}$$

Thus f affects both LP revenue and trader execution. Formal analyses show how constant-function markets track reference prices through arbitrage and generalize beyond two-asset products [2], [3], [16].

Let $V_{\mathrm{LP}}(T)$ be the terminal marked-to-market LP position, $V_{\mathrm{H}}(T)$ the value of holding the original assets, and $R_f(T)$ fee income in the same numeraire. A value-based loss and its fee-adjusted form are

$$IL(T) = V_{\mathrm{H}}(T) - V_{\mathrm{LP}}(T), \tag{3}$$

$$L_{\mathrm{net}}(T) = IL(T) - R_f(T). \tag{4}$$

This accounting distinction matters: an adaptive fee may compensate LPs through revenue without eliminating the stale-price mechanism that produces adverse selection. LVR makes that mechanism explicit by comparing the AMM with continuous rebalancing at the reference price [6], and recent work has further quantified the specific profitability zones where fee accumulation can turn impermanent loss into a sustainable gain [17].

A static pool sets $f_t = f$. A dynamic policy instead evaluates

$$f_t = \mathcal{F}(s_t, z_t; \theta), \tag{5}$$

where s_t is observable on-chain state, z_t optional external information, and θ a parameter vector. Directional policies return $f_t^{0\to1}$ and $f_t^{1\to0}$ separately, for example charging more when a swap moves the pool farther from an external reference.

B. Dynamic-Fee Research

Dynamic policies use different proxies for adverse conditions. Volatility-based mechanisms react to price variability; inventory policies account for reserve composition; oracle-informed policies use pool-to-reference deviation; and flow-based rules use recent volume, trade size, or direction. Lebedeva *et al.* compare block-level, transaction-level, and idealized oracle-informed mechanisms and find improved LP outcomes under their simulation assumptions [10]. Baggiani *et al.* identify two regimes in which fees balance arbitrage deterrence against attracting noise traders [11]. Campbell *et al.* similarly show that volatility and competing-venue costs jointly determine fee choice [12]. Recent agent-based analysis suggests that dynamic schedules may improve hedged LP

profitability mainly by increasing compensation in toxic states, rather than mechanically eliminating LVR [13].

These works optimize or compare economic policies. Our contribution is complementary: a common implementation substrate for configuring, deploying, and replaying multiple fee rules as Uniswap v4 hooks. It enables a later controlled economic comparison while keeping the surrounding software path fixed.

The closest studies therefore occupy distinct levels. Formal CFMM work supplies the market foundation [2], [3]; LVR supplies an LP-risk benchmark [6]; and dynamic-fee studies supply candidate policies and economic hypotheses [10], [11], [13]. This paper operates at the systems level by making several rules deployable and observable through one protocol-specific path. Neither level replaces the other: a framework without controlled baselines cannot demonstrate economic benefit, while a fee model without reproducible implementation leaves integration effects unmeasured.

While protocols like Balancer V3, Curve V2, and Algebra Finance have introduced internal mechanisms for adaptive fees or volatility-based adjustments, these features are tightly coupled to their specific protocol architectures. They do not provide a unified, external management layer. Consequently, researchers and developers cannot easily deploy, configure, monitor, and compare multiple fee strategies across a standardized infrastructure. This framework fills that gap by providing a protocol-agnostic (within the Uniswap v4 ecosystem) external contour for strategy lifecycle management.

Table I condenses this distinction. Prior studies primarily contribute market models, LP-risk measures, or fee policies. The present work does not replace those results; it supplies the implementation and replay path needed to instantiate and compare policy families on Uniswap v4.

C. Uniswap v4 and External Data

Uniswap v4 stores pools in a singleton `PoolManager` and settles intermediate balance changes through flash accounting [14]. Hooks attach custom logic to initialization, liquidity, swap, and donation callbacks. Their enabled permissions are encoded in low-order address bits, so deployment must produce an address matching the declared callbacks [18], [19]. For dynamic pools, a hook can call `updateDynamicLPFee` or return a per-swap override from `beforeSwap` [15]. OpenZeppelin provides reusable bases for callback validation and fee management, including `BaseDynamicFee` and `BaseOverrideFee` [20].

Oracle-informed policies require a reference unavailable from the pool itself. Chainlink Data Feeds expose aggregated observations to contracts [21]. For two USD-denominated feeds, the external ratio is $r_t^{\text{ext}} = p_{0,t}/p_{1,t}$. Production logic must normalize decimals and reject invalid or stale rounds. In the framework, deterministic mocks reproduce historical observations; they validate integration logic, not the timing or fault behavior of a live oracle.

III. Framework Design

A. Architecture and Workflow

NOTE for US: Need to be discussed with YURY. application focus needed?

The framework converts a supported dynamic-fee policy and its parameters into an inspectable, deployable, and experimentally testable Uniswap v4 project. It consists of four cooperating layers. The *contract layer* contains the Solidity hooks, shared fee interface, and policy state. The *application layer* uses Flask and browser code to load supported templates, expose their parameters, display generated source, and initiate project export or simulation. The *deployment layer* uses Foundry scripts to deploy the selected hook together with the required Uniswap v4 contracts, tokens, feeds, and liquidity pool. Finally, the *experiment layer* replays market observations through `Simulation.t.sol`, records contract and pool outputs, and converts them into metrics and figures.

Figure 1 summarizes the complete workflow. A developer first selects an available hook template and supplies its parameters. The generator creates a configured Solidity contract and packages it as a self-contained Foundry project. Deployment scripts then initialize a local Forge network, deploy the Uniswap v4 infrastructure and hook, and create a new pool associated with that hook. During evaluation, historical observations update Chainlink-compatible mock feeds, simulated swaps invoke the deployed contract, and the resulting fees, balances, gas usage, and LP outcomes are recorded.

This separation allows different fee policies to reuse the same deployment, pool initialization, trader model, and result-processing path. Consequently, differences between experimental outcomes can be attributed to the fee policy rather than to independently developed supporting infrastructure. Each stage also produces an inspectable artifact, including configured Solidity source, deployment logs, transaction traces, and experiment outputs.

B. Configuration and Project Generation

Let C denote a supported Solidity hook template and let $\theta = [\theta_1, \ldots, \theta_n]$ denote its exposed parameters. The generator produces a configured contract

$$C_\theta = \mathcal{G}(C, \theta). \tag{6}$$

The current implementation substitutes the selected values into a copy of the template while leaving the canonical source unchanged. Parameters can include initial fees, update increments, fee bounds, volatility sensitivity, and other policy-specific constants. The generated source is displayed before export so that the developer can inspect how the selected values affect the contract.

The exported project contains the configured hook, required interfaces, deployment scripts, Foundry tests, dependency declarations, remappings, historical-data utilities, and execution instructions. Exporting the complete project, rather than only the generated contract, preserves the surrounding environment

TABLE I
POSITIONING RELATIVE TO REPRESENTATIVE AMM AND DYNAMIC-FEE RESEARCH.

Work	Primary contribution	Relationship to this framework
Angeris *et al.* [2], [3]	Formal analysis and optimization of constant-function markets	Supplies the AMM foundation, not hook tooling
Milionis *et al.* [6]	LVR formulation for adverse-selection cost	Supplies an LP-risk benchmark for future evaluation
Lebedeva *et al.* [10]	Block-, deal-, and oracle-informed fee policies	Supplies a policy family represented by the templates
Baggiani *et al.* [11]	Stochastic-control formulation of dynamic fees	Supplies theoretical fee-design guidance
This work	Configurable contracts, deployment, replay, project export, and visualization	Supplies shared Uniswap v4 experimentation infrastructure

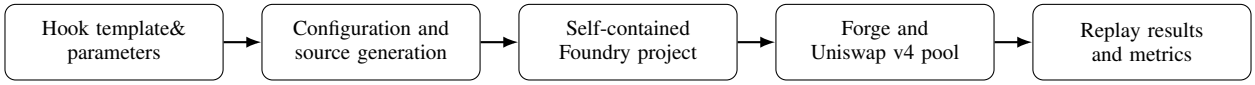

Fig. 1. Configuration, deployment, and evaluation workflow.

required to compile, deploy, and test the hook independently of the web interface.

The generator operates only on supported, allowlisted templates. It does not synthesize an arbitrary fee algorithm or prove that a selected parameter combination is economically or semantically valid. For example, incorrectly ordered fee bounds or incompatible oracle addresses may still produce an unsuitable configuration. Generated contracts must therefore be compiled, tested, and checked before deployment. A typed parameter schema with stronger cross-parameter validation remains future work.

C. Hook Lifecycle and Deployment

The hook implementations share a fee-oriented interface based on `BaseOverrideFee`. When a swap invokes a hook, the common execution path first verifies that the callback originates from the designated `PoolManager` and determines the swap direction. The hook then reads its policy state and, where required, an external price observation; computes and bounds the applicable fee; and returns the protocol-formatted fee override. After an executed swap, the hook updates only the state required by its policy. Some policies update after every trade, whereas others update at most once per block.

The comparative evaluation uses five dynamic implementations: `BAHook`, `DAHook`, `ABHook`, `VolatilityHook`, and `MEVChargeHook`. These policies differ in whether they respond to block-level external prices, individual swap direction, endogenous pool-price movement, recent price variance, or trade size. Their policy-specific equations and parameter values are described separately in the evaluated-policy section. The broader library also contains a peg-stability template and its associated tests, but that policy is not included in the comparative experiment.

Uniswap v4 encodes hook permissions in the low-order bits of the hook address. Deployment must therefore find a compatible `CREATE2` salt or use an equivalent deterministic procedure to obtain an address with the required permission pattern [19]. The deployment utilities create or resolve the `PoolManager`, position-management, and routing contracts; deploy mock tokens and price feeds; deploy the configured hook; construct the pool key; initialize the pool price; and provide the initial liquidity. The permission bitmap is checked before the pool is used because an incompatible address can prevent required callbacks from executing.

Historical Binance observations and Chainlink-compatible feeds serve different purposes. Binance provides the offline price and volume profiles used by the experiment, while the mock feeds expose those observations through the interface expected by oracle-dependent hooks. This arrangement tests contract integration deterministically; it does not reproduce the aggregation, update timing, latency, or failure behavior of a live Chainlink feed.

D. Testing and Reproducibility

Testing separates shared framework behavior from policy-specific behavior. Common Foundry tests verify deployment, hook permissions, pool initialization, callback authorization, liquidity provision, and successful swaps. Policy tests verify fee direction, update granularity, parameter bounds, and state transitions. The supplied project includes general hook tests and specialized suites such as `MEVChargeHookFees.t.sol` and `PegStabilityHook.t.sol`. Longer replay tests use `Simulation.t.sol` to apply market observations and collect fee, balance, gas, volume, and LP outputs.

A reproducible experiment must identify both the configured contract and the context in which it was executed. Each experimental cell is therefore represented by

$$E = \langle h(C_\theta), D, P_0, L_0, W, S \rangle, \tag{7}$$

where $h(C_\theta)$ is the configured-source hash, D identifies the compiler and dependency versions, P_0 and L_0 are the initial pool price and liquidity, W identifies the historical market window, and S contains the gas-price scenario, retail-demand level, and random seed. The historical input is additionally identified by a candle-data fingerprint.

All swaps are executed against local Uniswap v4 contracts in Foundry rather than calculated only by an off-chain model.

Contract gas is measured from these executions, and LP holdings are reconstructed from both pool state and trader balance changes. Reusing the same manifest, market window, and random inputs across policies enables paired comparisons while keeping the surrounding experimental conditions fixed.

IV. Experimental Evaluation

The evaluation addresses two questions: whether the framework can execute different fee policies through a common Uniswap v4 path, and whether any evaluated dynamic policy improves the LP outcome relative to a static fee under controlled market conditions.

A. Experimental Setup

One experimental cell is defined by a fee policy, trading pair, one-day market window, gas-price scenario, and retail-demand level. Each cell replays 1,440 one-minute Binance candles from 2024 against a locally deployed Uniswap v4 pool. The initial full-range position is valued at 20M USDT and divided equally between the two assets. The pool opens at the first external price, and a Chainlink-compatible mock feed is updated with each candle close. The first 60 minutes are executed as warm-up and excluded from measurement.

We study ETH/SHIB and ETH/USDC as volatile pairs and USDC/USDT as a stable pair. For every pair, complete days are ranked by daily realized volatility. Using one-minute closing prices ($P_{d,c}$), realized volatility is calculated as

$$RV_d = \sqrt{\sum_{c=2}^{1440} \left[\ln\left(\frac{P_{d,c}}{P_{d,c-1}} \right) \right]^2}. \tag{8}$$

The ranked days are divided into low-, medium-, and high-volatility terciles, from which 24 windows per tercile are retained. The resulting dataset contains 72 windows per pair and 216 pair-window combinations overall. Exact dates and candle fingerprints are stored with the experimental artifact.

Every swap is executed in the Foundry EVM against the deployed pool and hook. A calibration run measures median gas consumption for each policy rather than assigning an assumed cost. These measurements are then used in trader decisions under gas prices of 5, 20, and 80 gwei. The experimental manifest defined in Eq. (7) records the configured-source hash, dependency versions, initial state, market window, gas price, demand level, and random seed. As an internal accounting check, LP holdings are reconstructed independently from pool state and trader balance changes; the two calculations agree within a relative error of approximately 10^{-16}.

B. Trader and Demand Model

Order flow consists of simulated ordinary users and one gas-aware arbitrageur. These agents generate transactions against the historical price path; Binance candles provide external market conditions rather than individual Binance trades.

1) Arbitrageur: Let P be the external price, f the applicable swap fee, and $\gamma = 1 - f$. The fee creates a no-arbitrage interval $[P\gamma, P/\gamma]$. After ordinary-user flow in each minute, the arbitrageur moves the pool to the nearest interval boundary when the maximum expected profit exceeds transaction cost. For liquidity L, current square-root price s, and target square-root price t, the maximum profits in the two directions are

$$\Pi^*_{0\to1} = \frac{L(s-t)^2}{s}, \qquad \Pi^*_{1\to0} = \frac{L(t-s)^2}{\gamma s}. \tag{9}$$

A trade executes only when $\Pi^* > g p_{\text{gas}}$, where g is the measured gas consumption and p_{gas} is the scenario gas price. Because the fee produced by `MEVChargeHook` depends on trade size, its profit-maximizing size is found numerically using ternary search.

2) Ordinary users: For a candidate trade, V_{in} and V_{out} denote the input and output values at the external price, while G denotes gas cost. The normalized user outcome is

$$r = \frac{V_{\text{out}} - V_{\text{in}} - G}{V_{\text{out}} + V_{\text{in}} + G}. \tag{10}$$

A non-negative outcome is always accepted. A loss-making trade is accepted probabilistically:

$$P_{\text{exec}} = \begin{cases} 1, & r \geq 0, \\ e^{-\lambda |r|}, & r < 0. \end{cases} \tag{11}$$

We set $\lambda \approx 461.4$, which gives a representative trade with a 30-bps total cost an execution probability of approximately 0.5. This value is a behavioral assumption rather than an estimate from individual Binance users.

For every minute c, one candidate trade is generated independently in each direction:

$$q(c) = \frac{v_0(c)}{2} e^{\sigma Z - \sigma^2/2}, \qquad Z \sim \mathcal{N}(0,1), \quad \sigma = 1. \tag{12}$$

The lognormal multiplier generates heterogeneous trade sizes while preserving their expected aggregate value. Each candidate trade is executed in full or rejected according to Eq. (11).

The minute-level potential demand is

$$v_0(c) = \kappa B \hat{s}(c), \tag{13}$$

where $B = 20$M USDT is the initial pool value and $\hat{s}(c)$ is the normalized 2024 intraday Binance volume profile. Because $\sum_c \hat{s}(c) = 1$, κB is the potential daily retail demand. The five κ values vary demand relative to pool value without changing its intraday shape.

The reported arbitrage shares are simulation outputs, not externally imposed values. Low κ produces little ordinary-user flow relative to liquidity, making arbitrage a larger fraction of executed volume. The $\kappa = 1$ scenario is used as the reference point, while all conclusions are checked separately at the remaining levels.

TABLE II
RETAIL-DEMAND SCENARIOS AND ARBITRAGE SHARE UNDER THE 30-BPS BASELINE.

κ	Potential demand/day	Arbitrage share
0.03	0.6M USDT	89%
0.10	2M USDT	58%
0.30	6M USDT	25%
1.00	20M USDT	13%
3.00	60M USDT	16%

C. Fee Policies

The experiment compares five dynamic policies with static fees of 5, 30, 60, and 100 bps. The 30-bps policy is the predeclared primary baseline. All policies use the same deployment, pool, trader, and accounting path. Table III summarizes the implemented rules and their measured median gas consumption, including the intrinsic transaction cost.

The complete factorial design is

$$9 \text{ policies} \times 3 \text{ pairs} \times 72 \text{ windows} \times 3 \text{ gas levels} \times 5 \text{ demand levels} = 29{,}160 \text{ cells}. \quad (14)$$

D. Outcome and Statistical Comparison

The LP outcome for a cell is measured relative to holding the initially deposited assets:

$$Y(T) = V_{\mathrm{LP}}(T) + R_f(T) - V_{\mathrm{H}}(T) = R_f(T) - IL(T), \quad (15)$$

where $V_{\mathrm{LP}}(T)$ is the terminal LP position, $R_f(T)$ is fee income, and $V_{\mathrm{H}}(T)$ is the terminal value of holding the initial assets. All components are valued in USDT at the end-of-window external price.

Comparisons are paired. A policy is differenced against the 30-bps baseline using the same pair, window, gas price, demand level, and random inputs. This removes between-day variation unrelated to the fee rule. Effects are reported as median paired differences with 95% percentile-bootstrap confidence intervals obtained from 10,000 seeded resamples. A policy is classified as better or worse only when the interval excludes zero. Results from different κ levels are analyzed separately.

E. Results

1) Aggregate performance: Table IV presents the primary comparison at $\kappa = 1$. When results are aggregated across volatility regimes and gas scenarios, no dynamic policy significantly outperforms the 30-bps baseline. The confidence interval for `BAHook` includes zero, while the other four dynamic policies produce significantly lower outcomes. The exploratory 60-bps comparison is statistically indistinguishable from 30 bps.

The static policies produce mean outcomes of approximately 1.6k, 11.9k, 11.9k, and 7.9k USDT per window at 5, 30, 60, and 100 bps, respectively. Thus, under the assumed user-participation model, the static-fee curve contains a broad optimum around 30–60 bps rather than improving monotonically with the fee.

Variation in LP outcome is driven primarily by fee income and retained trading volume. The impermanent-loss component differs by only single-digit USDT across policies because the modeled arbitrageur moves terminal pool prices close to the external price. Under these assumptions, adaptation changes how flow is priced but does not materially change terminal inventory loss.

2) Conditional performance: Although no policy wins in aggregate, conditional advantages appear in high-volatility, low-gas scenarios. At $\kappa = 1$ and 5 gwei, `BAHook` improves the median LP outcome by (1,556) USDT per high-volatility window, with a 95% confidence interval of ([894,2,501]). Its advantage in medium-volatility windows is (367,[115,571]) USDT.

The policy associated with this advantage changes with order-flow composition. At $\kappa = 0.1$, the volatility hook and `DAHook` outperform the baseline in high-volatility, 5-gwei windows by (308,[32,742]) and (246,[59,563]) USDT, respectively. At $\kappa = 0.03$, the volatility hook retains an advantage in high-volatility windows across all three gas scenarios. These subgroup results are exploratory and should not be interpreted as universal policy rankings.

3) Gas overhead: Gas is a first-order determinant of the results. Dynamic hooks consume 88–123k gas per swap, compared with 76–77k for the static implementation. This additional cost enters ordinary-user acceptance and arbitrage profitability. At 80 gwei, every dynamic policy either loses to or is statistically indistinguishable from the baseline. Even the static implementation costs approximately $19 per swap in that scenario, equivalent to about 47 bps for a representative $4,000 trade. As gas increases from 5 to 80 gwei at $\kappa = 1$, the arbitrage share of executed volume rises from 12.4% to 13.7% because smaller ordinary-user trades are rejected first.

Overall, the results show that dynamic fees provide no unconditional advantage over a well-chosen static fee. They can improve LP outcomes in specific high-volatility, low-gas settings, but the relevant policy depends on order-flow composition and its advantage can be eliminated by additional execution cost.

F. Threats to Validity

The evaluation uses historical market observations but simulated order flow. Ordinary users follow a probabilistic acceptance rule and have no strategic or correlated behavior. One perfectly informed arbitrageur acts once per minute without mempool competition, latency, transaction reordering, or competing arbitrageurs. These assumptions simplify real DEX microstructure.

Liquidity is passive and full-range, and the initial position is fixed at 20M USDT. The demand scale κ is varied for sensitivity analysis but is not calibrated to individual Uniswap pools. One candidate trade per direction per minute produces coarser order flow than a transaction-level replay. The main matrix also uses a single recorded random seed.

TABLE III
FEE POLICIES INCLUDED IN THE COMPARATIVE EVALUATION.

Policy	Fee rule	Fee range	Gas
Static	Constant and identical in both directions	5, 30, 60, or 100 bps	76–77k
`DAHook`	After a swap, raises the fee in the executed direction by 1 bps and lowers the opposite fee by 1 bps	1–100 bps	88k
`BAHook`	Uses the external price-ratio direction and reallocates 5 bps between swap directions at most once per block	1–100 bps	108k
`ABHook`	Responds to the pool-price movement while maintaining a constant 60-bps sum across the two directions	1–59 bps per side	94k
Volatility hook	Applies the same fee in both directions using an exponentially weighted estimate of external-price variance	1–100 bps	123k
`MEVChargeHook`	Adds trade-size and recent-sender surcharges to a 30-bps base fee	30–100 bps	95k

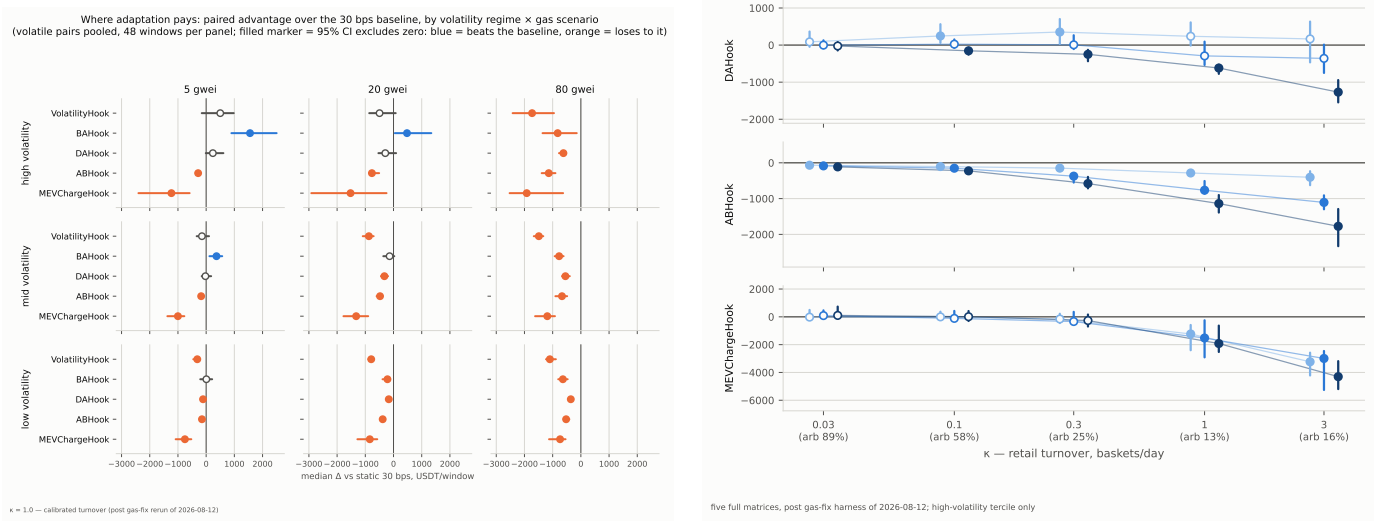

Fig. 2. Conditional policy performance. Left: paired advantage over the 30-bps baseline by volatility regime and gas price at $\kappa = 1$. Right: high-volatility performance across demand levels and gas prices. Filled markers indicate 95% confidence intervals that exclude zero.

The market data cover one year, three pairs, and one centralized exchange. The USDC/USDT scenario uses a synthetic unit feed for one leg. Chainlink-compatible mocks validate the hook interface but not live-oracle aggregation, latency, update thresholds, or failures. Local Foundry execution measures contract gas deterministically but does not reproduce public-chain congestion or routing competition.

Finally, the 30-bps baseline was selected before aggregation, whereas the conditional high-volatility findings were identified after examining the regime-level results and are therefore exploratory. The conclusions establish behavior under the declared model and should not be interpreted as evidence that the same policy will perform identically in a production pool.

V. SECURITY, VALIDITY, AND DISCUSSION

A. *Security Considerations*

Dynamic-fee hooks execute inside the swap lifecycle and enlarge the trusted code surface. Production implementations should restrict setters, accept callbacks only from the designated `PoolManager`, enforce protocol and policy fee

TABLE IV
PAIRED ADVANTAGE OVER THE 30-BPS BASELINE AT $\kappa = 1$, AGGREGATED OVER 18 VOLATILE-PAIR STRATA. VALUES ARE USDT PER WINDOW.

Policy	Median Δ	95% CI
Static 60 bps	(-2)	([-186,266])
`BAHook`	(-100)	([-453,350])
`DAHook`	(-235)	([-381,-104])
`ABHook`	(-460)	([-630,-288])
Volatility hook	(-752)	([-1069,-280])
`MEVChargeHook`	(-913)	([-1659,-728])

bounds, normalize oracle decimals, reject stale or nonpositive rounds, and handle division and casts safely. Address-permission validation is essential. Uniswap's security guidance emphasizes that incorrectly encoded permissions or custom deltas can produce unintended execution paths [22].

The web layer crosses a separate trust boundary. Hook names should be allowlisted; parameter types and ranges should be validated before substitution; generated files should be written to isolated temporary locations; and simulator arguments must not become shell fragments. Resource limits are needed because compilation and replay are user-triggered operations. Exported projects should pin revisions to prevent later dependency changes from silently altering a result.

Between the contract and application layers sits a data boundary: the implementation must check feed identity, sign, decimals, and timestamp, and define behavior for unavailable observations. Contract- and data-boundary threats admit code-level tests; application-layer risks and historical-replay bias instead require common baselines, multiple regimes, and explicit experimental reporting.

B. Validity and Design Trade-offs

Local execution provides deterministic debugging but not public-chain market microstructure. Historical replay omits mempool visibility, block-builder ordering, latency, gas competition, strategic arbitrage, and endogenous routing. Results also depend on initial price and liquidity, token decimals, observation frequency, trade sizes, valuation time, and the share of informed flow. One pair and interval cannot establish generality, particularly across volatile, stablecoin, and shallow-liquidity markets.

The framework's main benefit is separation of concerns: researchers can change $\mathcal{F}$ in Eq. (5) while retaining deployment, ingestion, and analysis. Generated source remains inspectable, and mocks make integration tests repeatable. The trade-off is that textual template substitution is less safe and expressive than a typed policy language; mock feeds validate calls but not production oracle behavior; and console parsing is less robust than a structured event-level result format.

Developing robust backtesting infrastructure for AMM strategies is an active area of research [23]. A controlled economic study within such a framework should hold the trace, initial reserves, liquidity range, valuation convention, and random seed constant while varying only the policy. It should compare all hooks with static-fee baselines and report LP value, fee income, IL or LVR, net outcome, retained volume, mean and distribution of fees, gas overhead, and runtime. Multiple pairs, volatility regimes, and repeated windows are required for uncertainty estimates. These additions are future evaluation work, not evidence inferred from the current trace.

The next implementation steps are therefore a typed parameter schema, containerized and version-pinned experiments, event-level result export, property-based tests for fee bounds and callback authorization, stale-oracle fault injection, and testnet validation. Together they would turn the current development framework into a stronger comparative research testbed. Furthermore, the framework is designed to easily accommodate emerging fee paradigms, such as trade-size-aware Impermanent-Loss Trimming (ILT) mechanisms and path-independent fee structures that preserve composability guaranties under transaction fragmentation [24].

VI. CONCLUSION

This paper presented an end-to-end framework for configuring, deploying, and replay-testing dynamic-fee hooks on Uniswap v4. Shared contracts and scripts isolate policy logic from protocol bootstrap; the web interface exposes parameters and source; and the experiment path connects historical observations to local execution and metrics. A 1,492-swap ETH/SHIB trace demonstrates that the integrated workflow operates. It does not prove that a dynamic policy outperforms static fees. The contribution is transparent, reusable infrastructure that enables such policies to be implemented and, with controlled baselines and broader traces, compared reproducibly.

REFERENCES

[1] Sam M. Werner, Daniel Perez, Lewis Gudgeon, Ariah Klages-Mundt, Dominik Harz, and William J. Knottenbelt. SoK: Decentralized finance (DeFi). In *Proc. 4th ACM Conf. Advances in Financial Technologies*, pages 30–46, 2022.

[2] Guillermo Angeris, Hsien-Tang Kao, Rei Chiang, Charlie Noyes, and Tarun Chitra. An analysis of uniswap markets, 2019.

[3] Guillermo Angeris, Akshay Agrawal, Alex Evans, Tarun Chitra, and Stephen Boyd. Constant function market makers: Multi-asset trades via convex optimization. In *Handbook on Blockchain*, pages 415–444. Springer, 2022.

[4] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. White paper, 2020.

[5] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. White paper, 2021.

[6] Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. Automated market making and loss-versus-rebalancing, 2022.

[7] Zhou Fan, Francisco Marmolejo-Cossio, Daniel J. Moroz, Michael Neuder, Rithvik Rao, and David C. Parkes. Strategic liquidity provision in uniswap v3, 2021.

[8] Roman Vlasov, Vladimir Gorgadze, and Artem Barger. The Impact of the Exchange Fees on Impermanent Loss of Liquidity Providers for Conservative Automated Market Makers. *The Journal of The British Blockchain Association*, may 27 2025.

[9] Robin Fritsch. A note on optimal fees for constant function market makers. In *Proceedings of the 2021 ACM CCS Workshop on Decentralized Finance and Security*, CCS '21, page 9–14. ACM, November 2021.

[10] Irina Lebedeva, Dmitrii Umnov, Yury Yanovich, Ignat Melnikov, and George Ovchinnikov. Dynamic fee for reducing impermanent loss in decentralized exchanges, 2025.

[11] Leonardo Baggiani, Martin Herdegen, and Leandro Sánchez-Betancourt. Optimal dynamic fees in automated market makers, 2025.

[12] Steven Campbell, Philippe Bergault, Jason Milionis, and Marcel Nutz. Optimal fees for liquidity provision in automated market makers, 2025.
[13] Daniele Maria Di Nosse and Fabrizio Lillo. Mitigating adverse selection in concentrated liquidity AMMs with dynamic fees: An agent-based model approach, 2026.
[14] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v4 core. White paper, 2024.
[15] Uniswap Labs. Dynamic fees. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[16] Guillermo Angeris and Tarun Chitra. Improved price oracles: Constant function market makers. In *Proc. 2nd ACM Conf. Advances in Financial Technologies*, pages 80–91, 2020.
[17] Ignat Melnikov, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. From impermanent loss to sustainable gain: Quantifying profitability zones for liquidity providers on dex. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2026. to appear.
[18] Uniswap Labs. Hooks. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[19] Uniswap Labs. Hook deployment. Uniswap v4 Developer Documentation, 2026. Accessed: 2026-07-17.
[20] OpenZeppelin. Uniswap hooks. Developer documentation, 2026. Accessed: 2026-07-17.
[21] Chainlink Labs. Chainlink data feeds. Developer documentation, 2026. Accessed: 2026-07-17.
[22] Uniswap Labs. Uniswap v4 security framework. Developer documentation, 2026. Accessed: 2026-07-17.
[23] Andrey Urusov, Rostislav Berezovskiy, and Yury Yanovich. Backtesting framework for concentrated liquidity market makers on uniswap v3 decentralized exchange. *Blockchain: Research and Applications*, 6(1):100256, 2025.
[24] Andrey Voronin, Roman Vlasov, Vladimir Gorgadze, Andrey Seoev, and Yury Yanovich. Characterizing path-independent fees: A route to zero impermanent loss in cpmms. In *2026 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–7, 2026.